

Cluster Analysis

Cluster Analysis:

- The process of grouping a set of physical or abstract objects into classes of similar objects is called clustering.
- A cluster is a collection of data objects that are similar to one another within the same cluster and are dissimilar to the objects in other clusters.
- A cluster of data objects can be treated collectively as one group and so may be considered as a form of data compression.
- Cluster analysis tools based on k-means, k-medoids, and several methods have also been built into many statistical analysis software packages or systems, such as S-Plus, SPSS, and SAS.

Applications:

- Cluster analysis has been widely used in numerous applications, including market research, pattern recognition, data analysis, and image processing.
- In business, clustering can help marketers discover distinct groups in their customer bases and characterize customer groups based on purchasing patterns.
- In biology, it can be used to derive plant and animal taxonomies, categorize genes with similar functionality, and gain insight into structures inherent in populations.
- Clustering may also help in the identification of areas of similar land use in an earth observation database and in the identification of groups of houses in a city according to house type, value, and geographic location, as well as the identification of groups of automobile insurance policy holders with a high average claim cost.
- Clustering is also called data segmentation in some applications because clustering partitions large data sets into groups according to their *similarity*.
- Clustering can also be used for outlier detection. Applications of outlier detection include the detection of credit card fraud and the monitoring of criminal activities in electronic commerce.

Typical Requirements Of Clustering In Data Mining:

➤ **Scalability:**

Many clustering algorithms work well on small data sets containing fewer than several hundred data objects; however, a large database may contain millions of objects. Clustering on a sample of a given large data set may lead to biased results.

Highly scalable clustering algorithms are needed.

➤ **Ability to deal with different types of attributes:**

Many algorithms are designed to cluster interval-based (numerical) data. However, applications may require clustering other types of data, such as binary, categorical (nominal), and ordinal data, or mixtures of these data types.

➤ **Discovery of clusters with arbitrary shape:**

Many clustering algorithms determine clusters based on Euclidean or Manhattan distance measures. Algorithms based on such distance measures tend to find spherical clusters with similar size and density.

However, a cluster could be of any shape. It is important to develop algorithms that can detect clusters of arbitrary shape.

➤ **Minimal requirements for domain knowledge to determine input parameters:**

Many clustering algorithms require users to input certain parameters in cluster analysis (such as the number of desired clusters). The clustering results can be quite sensitive to input parameters. Parameters are often difficult to determine, especially for data sets containing high-dimensional objects. This not only burdens users, but it also makes the quality of clustering difficult to control.

➤ **Ability to deal with noisy data:**

Most real-world databases contain outliers or missing, unknown, or erroneous data.

Some clustering algorithms are sensitive to such data and may lead to clusters of poor quality.

➤ **Incremental clustering and insensitivity to the order of input records:**

Some clustering algorithms cannot incorporate newly inserted data (i.e., database updates) into existing clustering structures and, instead, must determine a new clustering from scratch. Some clustering algorithms are sensitive to the order of input data.

That is, given a set of data objects, such an algorithm may return dramatically different clusterings depending on the order of presentation of the input objects.

It is important to develop incremental clustering algorithms and algorithms that are insensitive to the order of input.

➤ **High dimensionality:**

A database or a data warehouse can contain several dimensions or attributes. Many clustering algorithms are good at handling low-dimensional data, involving only two to three dimensions. Human eyes are good at judging the quality of clustering for up to three dimensions. Finding clusters of data objects in high-dimensional space is challenging, especially considering that such data can be sparse and highly skewed.

➤ **Constraint-based clustering:**

Real-world applications may need to perform clustering under various kinds of constraints. Suppose that your job is to choose the locations for a given number of new automatic banking machines (ATMs) in a city. To decide upon this, you may cluster households while considering constraints such as the city's rivers and highway networks, and the type and number of customers per cluster. A challenging task is to find groups of data with good clustering behavior that satisfy specified constraints.

➤ **Interpretability and usability:**

Users expect clustering results to be interpretable, comprehensible, and usable. That is, clustering may need to be tied to specific semantic interpretations and applications. It is important to study how an application goal may influence the selection of clustering features and methods.

Major Clustering Methods:

- Partitioning Methods
- Hierarchical Methods
- Density-Based Methods
- Grid-Based Methods
- Model-Based Methods

Types Of Data in Clustering

Types Of Data Used In Cluster Analysis Are:

- Interval-Scaled variables
- Binary variables
- Nominal, Ordinal, and Ratio variables
- Variables of mixed types

Types Of Data Structures

Main memory-based clustering algorithms typically operate on either of the following two data structures.

- Data Matrix (or object by variable structure)
- Dissimilarity Matrix (or object by object structure)

Data Matrix

This represents n objects, such as persons, with p variables (also called measurements or attributes), such as age, height, weight, gender, race and so on. The structure is in the form of a relational table, or n -by- p matrix (n objects $\times$ p variables)

The Data Matrix is often called a two-mode matrix since the rows and columns of this represent the different entities.

$$\begin{bmatrix} x_{11} & \dots & x_{1f} & \dots & x_{1p} \\ \dots & \dots & \dots & \dots & \dots \\ x_{i1} & \dots & x_{if} & \dots & x_{ip} \\ \dots & \dots & \dots & \dots & \dots \\ x_{n1} & \dots & x_{nf} & \dots & x_{np} \end{bmatrix}$$

Dissimilarity Matrix

This stores a collection of proximities that are available for all pairs of n objects. It is often represented by a n – by – n table, where $d(i,j)$ is the measured difference or dissimilarity between objects i and j . In general, $d(i,j)$ is a non-negative number that is close to 0 when objects i and j are highly similar or “near” each other and becomes larger the more they differ. Since $d(i,j) = d(j,i)$ and $d(i,i) = 0$, we have the matrix in figure.

This is also called as one mode matrix since the rows and columns of this represent the same entity.

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \dots & \dots & 0 \end{bmatrix}$$

Types Of Data

Interval-Scaled Variables

Interval-scaled variables are continuous measurements of a roughly linear scale.

Typical examples include weight and height, latitude and longitude coordinates (e.g., when clustering houses), and weather temperature.

The measurement unit used can affect the clustering analysis. For example, changing measurement units from meters to inches for height, or from kilograms to pounds for weight, may lead to a very different clustering structure.

In general, expressing a variable in smaller units will lead to a larger range for that variable, and thus a larger effect on the resulting clustering structure.

To help avoid dependence on the choice of measurement units, the data should be standardized. Standardizing measurements attempts to give all variables an equal weight.

This is especially useful when given no prior knowledge of the data. However, in some applications, users may intentionally want to give more weight to a certain set of variables than to others.

For example, when clustering basketball player candidates, we may prefer to give more weight to the variable height.

Binary Variables

A binary variable is a variable that can take only 2 values.

For example, generally, gender variables can take 2 variables male and female.

Contingency Table For Binary Data

Let us consider binary values 0 and 1

contingency table

	1	0	<i>sum</i>
1	<i>a</i>	<i>b</i>	<i>a+b</i>
0	<i>c</i>	<i>d</i>	<i>c+d</i>
<i>sum</i>	<i>a+c</i>	<i>b+d</i>	<i>p</i>

Let $p=a+b+c+d$

Simple matching coefficient (invariant, if the binary variable is symmetric):

$$d(i, j) = \frac{b + c}{a + b + c + d}$$

Jaccard coefficient (noninvariant if the binary variable is asymmetric):

$$d(i, j) = \frac{b + c}{a + b + c}$$

Nominal or Categorical Variables

A generalization of the binary variable in that it can take more than 2 states, e.g., red, yellow, blue, green.

Method 1: Simple matching

The dissimilarity between two objects i and j can be computed based on the simple matching.

m : Let m be no of matches (i.e., the number of variables for which i and j are in the same state).

p : Let p be total no of variables.

$$d(i, j) = \frac{p - m}{p}$$

Method 2: use a large number of binary variables

Creating a new binary variable for each of the M nominal states.

Ordinal Variables

An ordinal variable can be discrete or continuous.

In this order is important, e.g., rank.

It can be treated like interval-scaled

By replacing x_{if} by their rank,

$$r_{if} \in \{1, \dots, M_f\}$$

By mapping the range of each variable onto $[0, 1]$ by replacing the i -th object in the f -th variable by,

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

Then compute the dissimilarity using methods for interval-scaled variables.

Ratio-Scaled Intervals

Ratio-scaled variable: It is a positive measurement on a nonlinear scale, approximately at an exponential scale, such as Ae^{Bt} or Ae^{-Bt} .

Methods:

First, treat them like interval-scaled variables — not a good choice! (why?)

Then apply logarithmic transformation i.e. $y = \log(x)$

Finally, treat them as continuous ordinal data treat their rank as interval-scaled.

Variables Of Mixed Type

A database may contain all the six types of variables

symmetric binary, asymmetric binary, nominal, ordinal, interval, and ratio.

And those combinedly called as mixed-type variables.

Partitioning Methods:

A partitioning method constructs k partitions of the data, where each partition represents a cluster and $k \leq n$. That is, it classifies the data into k groups, which together satisfy the following requirements:

- Each group must contain at least one object, and
- Each object must belong to exactly one group.

A partitioning method creates an initial partitioning. It then uses an iterative relocation technique that attempts to improve the partitioning by moving objects from one group to another.

The general criterion of a good partitioning is that objects in the same cluster are close or related to each other, whereas objects of different clusters are far apart or very different.

Classical Partitioning Methods:

The most well-known and commonly used partitioning methods are

- ❖ The k -Means Method
- ❖ k -Medoids Method

Centroid-Based Technique: The K -Means Method:

The k -means algorithm takes the input parameter, k , and partitions a set of n objects into k clusters so that the resulting intra cluster similarity is high but the inter cluster similarity is low.

Cluster similarity is measured in regard to the mean value of the objects in a cluster, which can be viewed as the cluster's centroid or center of gravity.

The k -means algorithm proceeds as follows.

- First, it randomly selects k of the objects, each of which initially represents a cluster mean or center.
- For each of the remaining objects, an object is assigned to the cluster to which it is the most similar, based on the distance between the object and the cluster mean.
- It then computes the new mean for each cluster.
- This process iterates until the criterion function converges.

Typically, the square-error criterion is used, defined as

$$E = \sum_{i=1}^k \sum_{p \in C_i} |p - m_i|^2,$$

Where E is the sum of the square error for all objects in the data set p is the point in space representing a given object m_i is the mean of cluster C_i

The k-means partitioning algorithm:

The k -means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

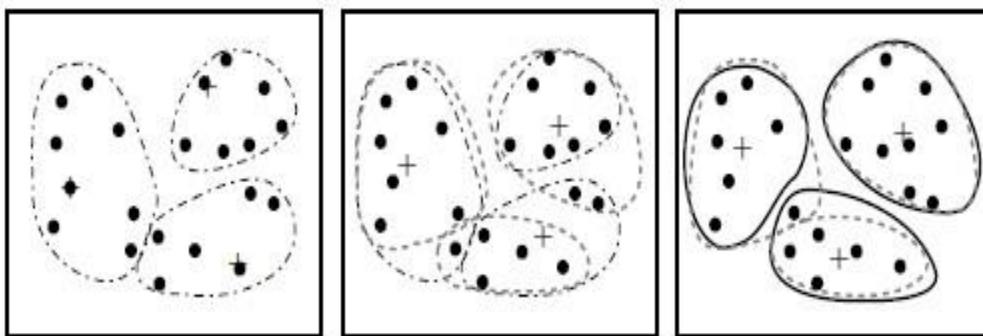
Input:

- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- (1) arbitrarily choose k objects from D as the initial cluster centers;
- (2) **repeat**
- (3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;
- (4) update the cluster means, i.e., calculate the mean value of the objects for each cluster;
- (5) **until** no change;

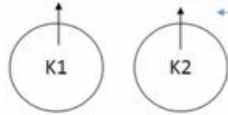


Clustering of a set of objects based on the k -means method

Sr. No.	Height	Weight	Clusters
1	185	72	K1
2	170	56	K2
3	168	60	K2
4	179	68	K1
5	182	72	?
6	188	77	?
7	180	71	?
8	180	70	?
9	183	84	?
10	180	88	?
11	180	67	?
12	177	76	?

STEP 1: Decide Centroids

K1: (185,72) K2:(170,56)



Example

STEP 2: Find shortest distance

$$K1 = \sqrt{(168 - 185)^2 + (60 - 72)^2}$$

$$K1 = 20.80$$

$$K2 = \sqrt{(168 - 170)^2 + (60 - 56)^2}$$

$$K2 = 4.48 \text{ (Minimum Dist)}$$

STEP 3: New Centroid Calculation

$$K2: (170+168/2, 60+56/2)$$

$$K2: (169,58) \text{ (New Centroid)}$$

✓ Euclidean Distance Formula:

$$d = \sqrt{(x_o - x_c)^2 + (y_o - y_c)^2}$$

- X_o, y_o = Observe Values
- X_c, y_c = Centroid Values
- d = distance

Updated Centroids

K1: (185,72) K2:(169,58)



STEP 4: Find shortest distance

$$K1 = \sqrt{(179 - 185)^2 + (68 - 72)^2}$$

$$K1 = 6.32 \text{ (Minimum Dist)}$$

$$K2 = \sqrt{(179 - 169)^2 + (68 - 58)^2}$$

$$K2 = 14.14$$

Remaining Steps:

Repeat Step 3 & Continue...

➤ **Clusters are: K=2**

K1= {1,4,5,6,7,8,9,10,11,12}

K2 = {2,3}

Advantages of K-means:

Simple and easy to implement.

Efficient for large datasets.

Converges quickly for small to medium-sized datasets.

Limitations of K-means:

Requires specifying the number of clusters (k) in advance.

Sensitive to initial centroid selection.

Assumes clusters are spherical and equally sized.

Struggles with clusters of different densities and non-linear shapes.

The k -Medoids Method:

- The k -means algorithm is sensitive to outliers because an object with an extremely large value may substantially distort the distribution of data. This effect is particularly exacerbated due to the use of the square-error function.
- Instead of taking the mean value of the objects in a cluster as a reference point, we can pick actual objects to represent the clusters, using one representative object per cluster. Each remaining object is clustered with the representative object to which it is the most similar. The partitioning method is then performed based on the principle of minimizing the sum of the dissimilarities between each object and its corresponding reference point. That is, an absolute-error criterion is used, defined as

$$E = \sum_{j=1}^k \sum_{p \in C_j} |p - o_j|,$$

Where E is the sum of the absolute error for all objects in the

data set p is the point in space representing a given object in

cluster C_j o_j is the representative object of C_j

- The initial representative objects are chosen arbitrarily. The iterative process of replacing representative objects by non representative objects continues as long as the quality of the resulting clustering is improved.
- This quality is estimated using a cost function that measures the average dissimilarity between an object and the representative object of its cluster.
- To determine whether a non representative object, o_j random, is a good replacement for a current representative object, o_j , the following four cases are examined for each of the nonrepresentative objects.

Case 1:

p currently belongs to representative object, o_j . If o_j is replaced by o_{random} as a representative object and p is closest to one of the other representative objects, $o_i, i \neq j$, then p is reassigned to o_i .

Case 2:

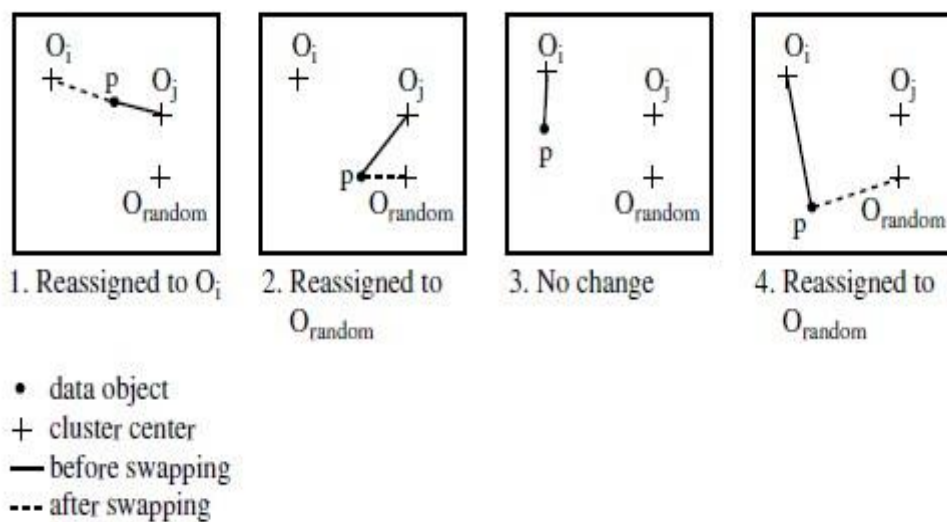
P currently belongs to representative object, o_j . If o_j is replaced by o_{random} as a representative object and p is closest to o_{random} , then p is reassigned to o_{random} .

Case 3:

P currently belongs to representative object, o_i , $i \neq j$. If o_j is replaced by o_{random} as a representative object and p is still closest to o_i , then the assignment does not change.

Case 4:

P currently belongs to representative object, o_i , $i \neq j$. If o_j is replaced by o_{random} as a representative object and p is closest to o_{random} , then p is reassigned to o_{random} .



Four cases of the cost function for k -medoids clustering

The k -Medoids Algorithm:

The k -medoids algorithm for partitioning based on medoid or central objects.

Input:

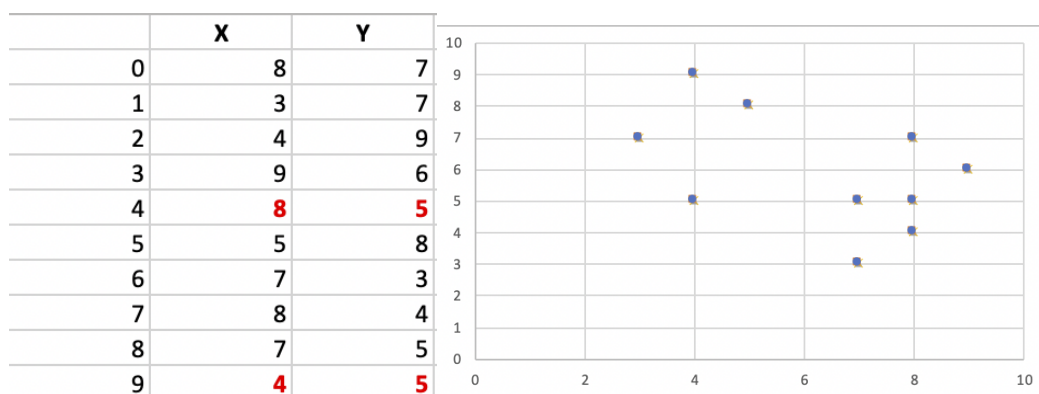
- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- (1) arbitrarily choose k objects in D as the initial representative objects or seeds;
- (2) **repeat**
- (3) assign each remaining object to the cluster with the nearest representative object;
- (4) randomly select a nonrepresentative object, o_{random} ;
- (5) compute the total cost, S , of swapping representative object, o_j , with o_{random} ;
- (6) **if** $S < 0$ **then** swap o_j with o_{random} to form the new set of k representative objects;
- (7) **until** no change;

The k -medoids method is more robust than k -means in the presence of noise and outliers, because a medoid is less influenced by outliers or other extreme values than a mean. However, its processing is more costly than the k -means method.



Step 1: Let the randomly selected 2 medoids, so select $k = 2$, and let C_1 -(4, 5) and C_2 -(8, 5) are the two medoids.

Step 2: Calculating cost. The dissimilarity of each non-medoid point with the medoids is calculated and tabulated:

$$\text{Distance} = |X_1 - X_2| + |Y_1 - Y_2|.$$

	X	Y	Dissimilarity from C1	Dissimilarity from C2
0	8	7	6	2
1	3	7	3	7
2	4	9	4	8
3	9	6	6	2
4	8	5	-	-
5	5	8	4	6
6	7	3	5	3
7	8	4	5	1
8	7	5	3	1
9	4	5	-	-

Each point is assigned to the cluster of that medoid whose dissimilarity is less. Points 1, 2, and 5 go to cluster C1 and 0, 3, 6, 7, 8 go to cluster C2. The Cost = $(3 + 4 + 4) + (3 + 1 + 1 + 2 + 2) = 20$

randomly select one non-medoid point and recalculate the cost. Let the randomly selected point be (8, 4). The dissimilarity of each non-medoid point with the medoids – C1 (4, 5) and C2 (8, 4) is calculated

	X	Y	Dissimilarity from C1	Dissimilarity from C2
0	8	7	6	3
1	3	7	3	8
2	4	9	4	9
3	9	6	6	3
4	8	5	4	1
5	5	8	4	7
6	7	3	5	2
7	8	4	-	-
8	7	5	3	2
9	4	5	-	-

points 1, 2, and 5 go to cluster C1 and 0, 3, 6, 7, 8 go to cluster C2. The New cost = $(3 + 4 + 4) + (2 + 2 + 1 + 3 + 3) = 22$

Swap Cost = New Cost – Previous Cost = $22 - 20 = 2 > 0$

As the swap cost is not less than zero, we undo the swap. Hence (4, 5) and (8, 5) are the final medoids.

Advantages:

1. It is simple to understand and easy to implement.
2. K-Medoid Algorithm is fast and converges in a fixed number of steps.
3. PAM is less sensitive to outliers than other partitioning algorithms.

Disadvantages:

1. The main disadvantage of K-Medoid algorithms is that it is not suitable for clustering non-spherical (arbitrarily shaped) groups of objects. This is because it relies on minimizing the distances between the non-medoid objects and the medoid (the cluster center) – briefly, it uses compactness as clustering criteria instead of connectivity.
2. It may obtain different results for different runs on the same dataset because the first k medoids are chosen randomly.

CLARA:

It is an extension to PAM to support Medoid clustering for large data sets. This algorithm selects **data samples from the data set, applies Pam on each sample, and**

outputs the best Clustering out of these samples. This is more effective than PAM. We should ensure that the selected samples aren't biased as they affect the Clustering of the whole data.

CLARANS:

This algorithm selects a sample of neighbors to examine instead of selecting samples from the data set. In every step, it examines the neighbors of every node.

Hierarchical_clustering

Hierarchical_clustering is a method of cluster analysis in data mining that creates a hierarchical representation of the clusters in a dataset. The method starts by treating each data point as a separate cluster and then iteratively combines the closest clusters until a stopping criterion is reached. The result of hierarchical clustering is a tree-like structure, called a dendrogram, which illustrates the hierarchical relationships among the clusters.

Hierarchical clustering has several advantages over other clustering methods

- The ability to handle non-convex clusters and clusters of different sizes and densities.
- The ability to handle missing data and noisy data.
- The ability to reveal the hierarchical structure of the data, which can be useful for understanding the relationships among the clusters.

Drawbacks of Hierarchical Clustering

- The need for a criterion to stop the clustering process and determine the final number of clusters.
- The computational cost and memory requirements of the method can be high, especially for large datasets.
- The results can be sensitive to the initial conditions, linkage criterion, and distance metric used.

In summary, Hierarchical clustering is a method of data mining that groups similar data points into clusters by creating a hierarchical structure of the clusters.

- This method can handle different types of data and reveal the relationships among the clusters. However, it can have high computational cost and results can be sensitive to some conditions.

Types of Hierarchical Clustering

Basically, there are two types of hierarchical Clustering:

1. Agglomerative Clustering
2. Divisive clustering

1. Agglomerative Clustering

Initially consider every data point as an **individual** Cluster and at every step, merge the nearest pairs of the cluster. (It is a bottom-up method). At first, every dataset is considered an individual entity or cluster. At every iteration, the clusters merge with different clusters until one cluster is formed.

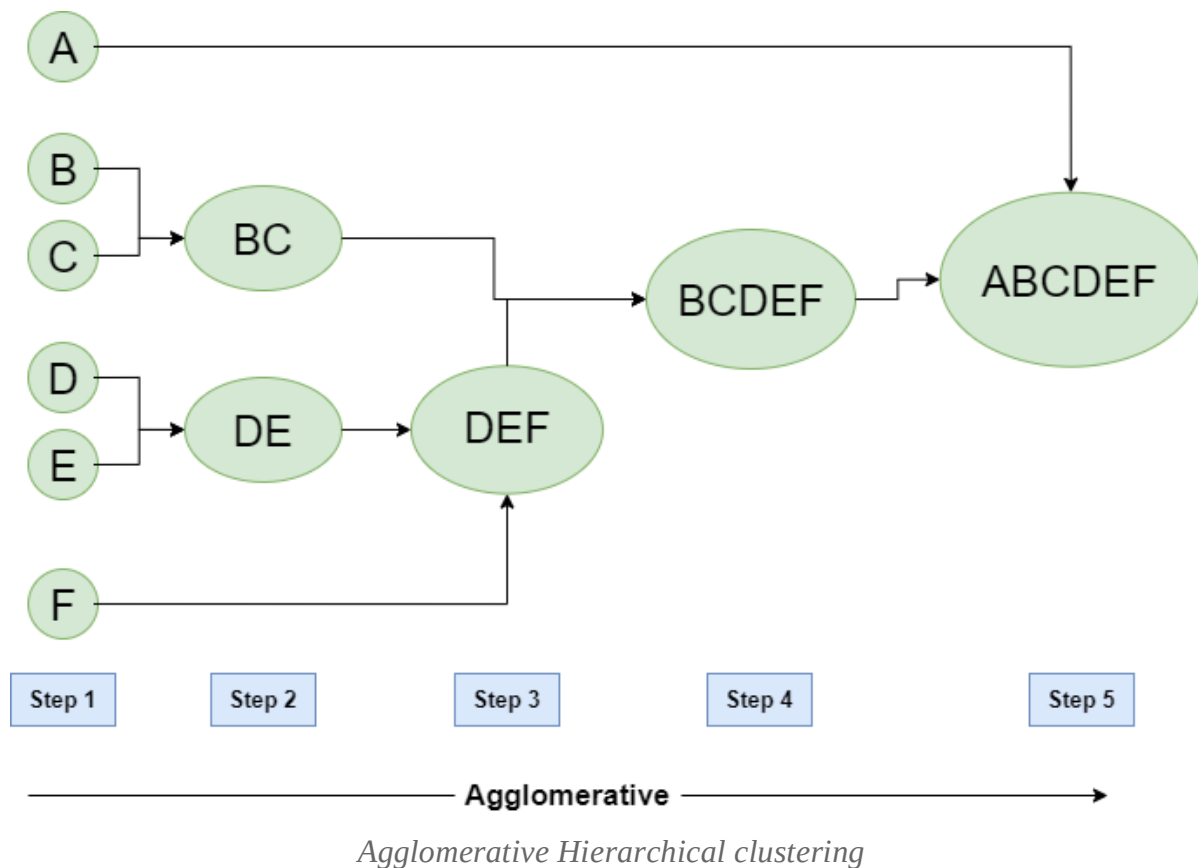
The algorithm for Agglomerative Hierarchical Clustering is:

- Calculate the similarity of one cluster with all the other clusters (calculate proximity matrix)
- Consider every data point as an individual cluster
- Merge the clusters which are highly similar or close to each other.
- Recalculate the proximity matrix for each cluster
- Repeat Steps 3 and 4 until only a single cluster remains.

Let's see the graphical representation of this algorithm using a dendrogram.

Note: This is just a demonstration of how the actual algorithm works no calculation has been performed below all the proximity among the clusters is assumed.

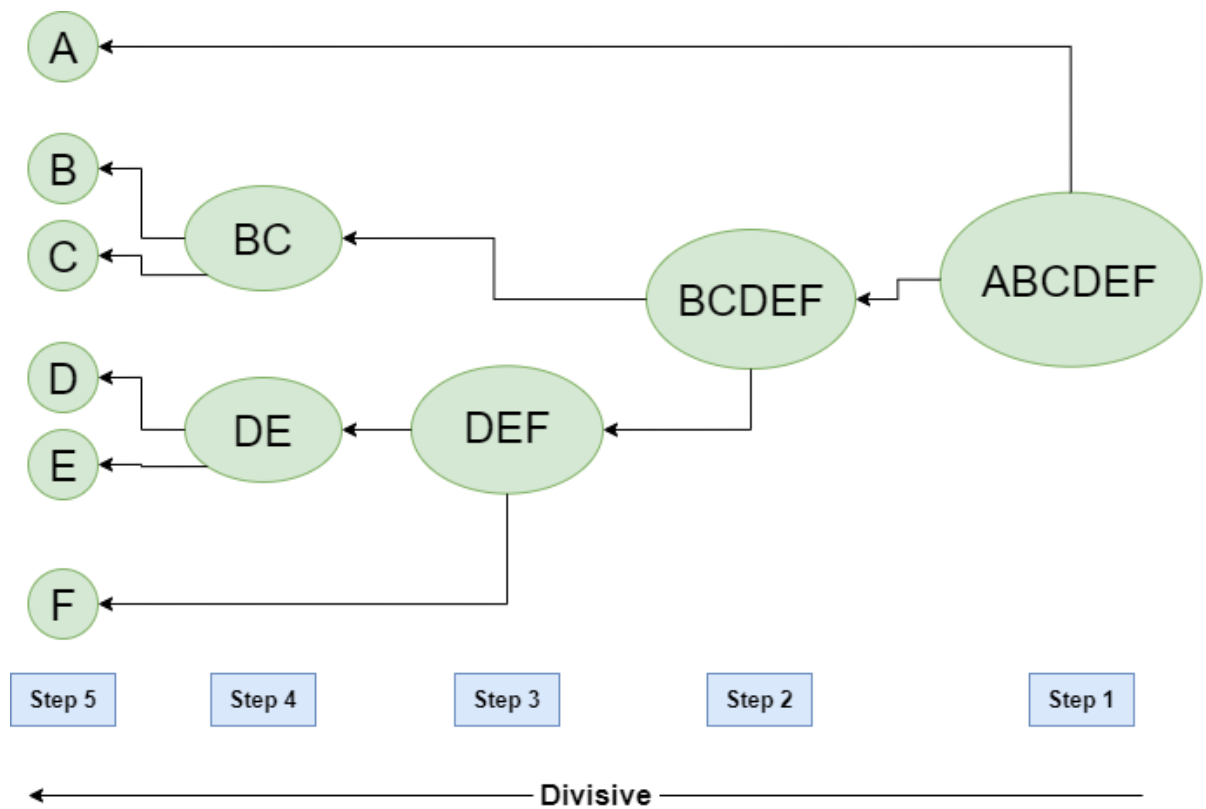
Let's say we have six data points **A, B, C, D, E, and F**.



- **Step-1:** Consider each alphabet as a single cluster and calculate the distance of one cluster from all the other clusters.
- **Step-2:** In the second step comparable clusters are merged together to form a single cluster. Let's say cluster (B) and cluster (C) are very similar to each other therefore we merge them in the second step similarly to cluster (D) and (E) and at last, we get the clusters [(A), (BC), (DE), (F)]
- **Step-3:** We recalculate the proximity according to the algorithm and merge the two nearest clusters([(DE), (F)]) together to form new clusters as [(A), (BC), (DEF)]
- **Step-4:** Repeating the same process; The clusters DEF and BC are comparable and merged together to form a new cluster. We're now left with clusters [(A), (BCDEF)].
- **Step-5:** At last, the two remaining clusters are merged together to form a single cluster [(ABCDEF)].

2. Divisive Hierarchical clustering

We can say that Divisive Hierarchical clustering is precisely the **opposite** of Agglomerative Hierarchical clustering. In Divisive Hierarchical clustering, we take into account all of the data points as a single cluster and in every iteration, we separate the data points from the clusters which aren't comparable. In the end, we are left with N clusters.



Hierarchical_clustering

Hierarchical_clustering is a method of cluster analysis in data mining that creates a hierarchical representation of the clusters in a dataset. The method starts by treating each data point as a separate cluster and then iteratively combines the closest clusters until a stopping criterion is reached. The result of hierarchical clustering is a tree-like structure, called a dendrogram, which illustrates the hierarchical relationships among the clusters.

Hierarchical clustering has several advantages over other clustering methods

- The ability to handle non-convex clusters and clusters of different sizes and densities.
- The ability to handle missing data and noisy data.
- The ability to reveal the hierarchical structure of the data, which can be useful for understanding the relationships among the clusters.

Drawbacks of Hierarchical Clustering

- The need for a criterion to stop the clustering process and determine the final number of clusters.
- The computational cost and memory requirements of the method can be high, especially for large datasets.
- The results can be sensitive to the initial conditions, linkage criterion, and distance metric used.

In summary, Hierarchical clustering is a method of data mining that groups similar data points into clusters by creating a hierarchical structure of the clusters.

- This method can handle different types of data and reveal the relationships among the clusters. However, it can have high computational cost and results can be sensitive to some conditions.

Types of Hierarchical Clustering

Basically, there are two types of hierarchical Clustering:

1. Agglomerative Clustering
2. Divisive clustering

1. Agglomerative Clustering

Initially consider every data point as an **individual** Cluster and at every step, merge the nearest pairs of the cluster. (It is a bottom-up method). At first, every dataset is considered an individual entity or cluster. At every iteration, the clusters merge with different clusters until one cluster is formed.

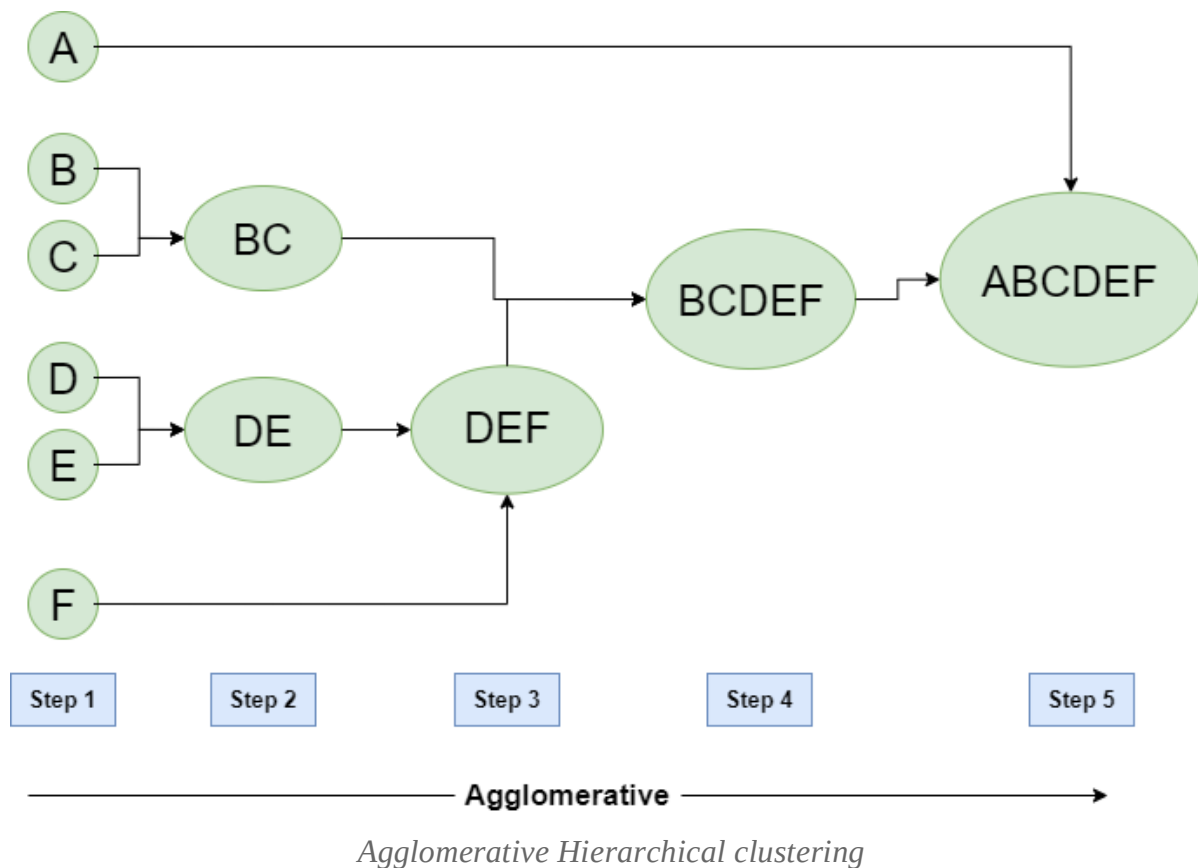
The algorithm for Agglomerative Hierarchical Clustering is:

- Calculate the similarity of one cluster with all the other clusters (calculate proximity matrix)
- Consider every data point as an individual cluster
- Merge the clusters which are highly similar or close to each other.
- Recalculate the proximity matrix for each cluster
- Repeat Steps 3 and 4 until only a single cluster remains.

Let's see the graphical representation of this algorithm using a dendrogram.

Note: This is just a demonstration of how the actual algorithm works no calculation has been performed below all the proximity among the clusters is assumed.

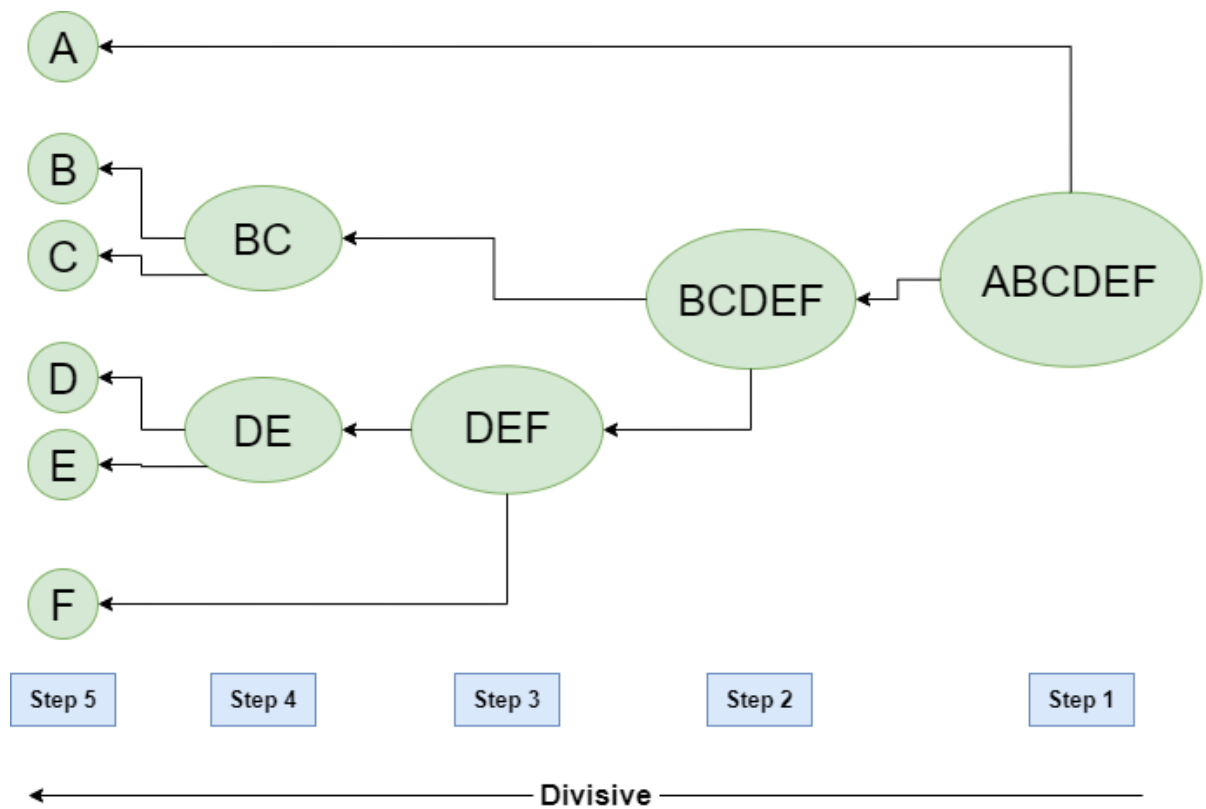
Let's say we have six data points **A, B, C, D, E, and F**.



- **Step-1:** Consider each alphabet as a single cluster and calculate the distance of one cluster from all the other clusters.
- **Step-2:** In the second step comparable clusters are merged together to form a single cluster. Let's say cluster (B) and cluster (C) are very similar to each other therefore we merge them in the second step similarly to cluster (D) and (E) and at last, we get the clusters [(A), (BC), (DE), (F)]
- **Step-3:** We recalculate the proximity according to the algorithm and merge the two nearest clusters([(DE), (F)]) together to form new clusters as [(A), (BC), (DEF)]
- **Step-4:** Repeating the same process; The clusters DEF and BC are comparable and merged together to form a new cluster. We're now left with clusters [(A), (BCDEF)].
- **Step-5:** At last, the two remaining clusters are merged together to form a single cluster [(ABCDEF)].

2. Divisive Hierarchical clustering

We can say that Divisive Hierarchical clustering is precisely the **opposite** of Agglomerative Hierarchical clustering. In Divisive Hierarchical clustering, we take into account all of the data points as a single cluster and in every iteration, we separate the data points from the clusters which aren't comparable. In the end, we are left with N clusters.



Density-Based Clustering

- ❖ Most partitioning methods cluster objects based on the distance between objects. Such methods can find only spherical-shaped clusters and encounter difficulty at discovering clusters of arbitrary shapes.
- ❖ Other clustering methods have been developed based on the notion of density. Their general idea is to continue growing the given cluster as long as the density in the neighborhood exceeds some threshold; that is, for each data point within a given cluster, the neighborhood of a given radius has to contain at least a minimum number of points. Such a method can be used to filter out noise (outliers) and discover clusters of arbitrary shape.
- ❖ DBSCAN and its extension, OPTICS, are typical density-based methods that grow clusters according to a density-based connectivity analysis. DENCLUE is a method that clusters objects based on the analysis of the value distributions of density functions.

Density-Based Clustering refers to unsupervised learning methods that identify distinctive groups/clusters in the data, based on the idea that a cluster in data space is a contiguous region of high point density, separated from other such clusters by contiguous regions of low point density.

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is a base algorithm for density-based clustering. It can discover clusters of different shapes and sizes from a large amount of data, which is containing noise and outliers.

The DBSCAN algorithm uses two parameters:

- **minPts:** The minimum number of points (a threshold) clustered together for a region to be considered dense.

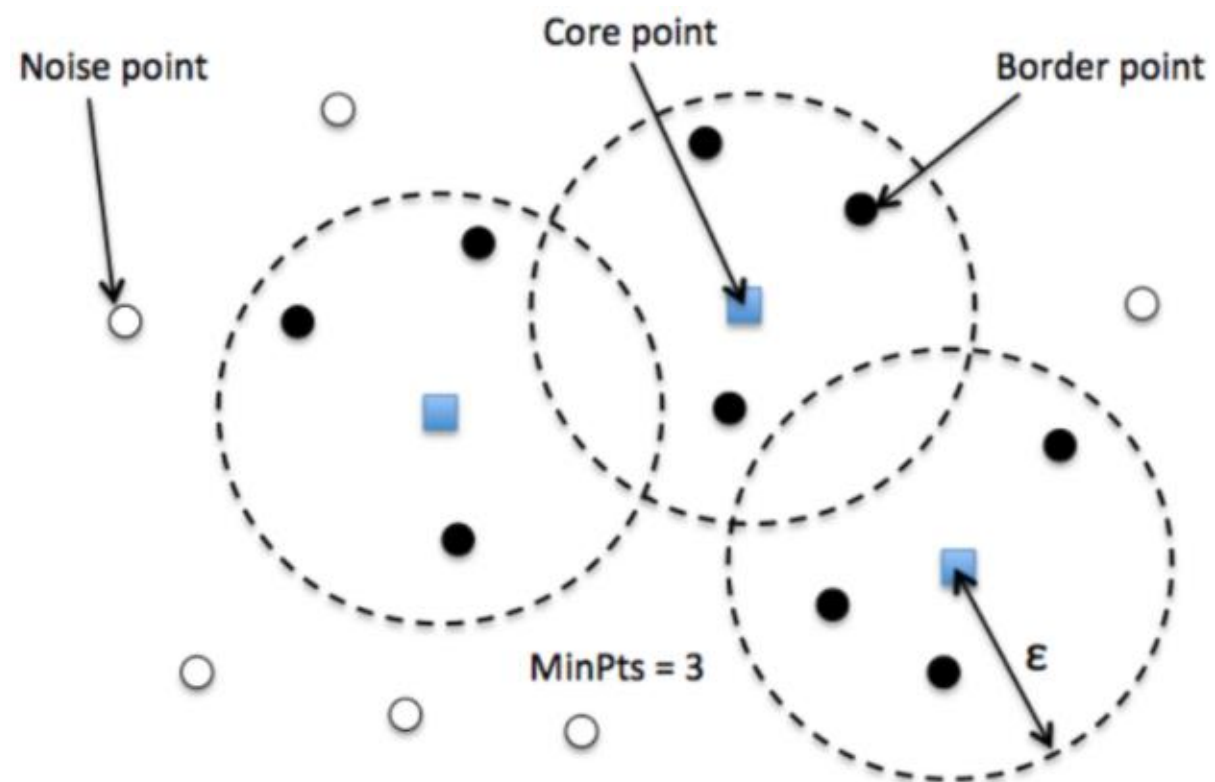
- **eps (ϵ):** A distance measure that will be used to locate the points in the neighborhood of any point.

These parameters can be understood if we explore two concepts called Density Reachability and Density Connectivity.

Reachability in terms of density establishes a point to be reachable from another if it lies within a particular distance (eps) from it.

Connectivity, on the other hand, involves a transitivity based chaining-approach to determine whether points are located in a particular cluster. For example, p and q points could be connected if $p \rightarrow r \rightarrow s \rightarrow t \rightarrow q$, where $a \rightarrow b$ means b is in the neighborhood of a.

There are three types of points after the DBSCAN clustering is complete:



- **Core** — This is a point that has at least m points within distance n from itself.
- **Border** — This is a point that has at least one Core point at a distance n .

- **Noise** — This is a point that is neither a Core nor a Border. And it has less than m points within distance n from itself.

Algorithmic steps for DBSCAN clustering

Let $X = \{x_1, x_2, x_3, \dots, x_n\}$ be the set of data points. DBSCAN requires two parameters: ϵ (eps) and the minimum number of points required to form a cluster (minPts).

1. Start with an arbitrary starting point that has not been visited.
2. Extract the neighborhood of this point using ϵ (All points which are within the ϵ distance are neighborhood).
3. If there are sufficient neighborhood around this point then clustering process starts and point is marked as visited else this point is labeled as noise (Later this point can become the part of the cluster).
4. If a point is found to be a part of the cluster then its ϵ neighborhood is also the part of the cluster and the above procedure from step 2 is repeated for all ϵ neighborhood points. This is repeated until all points in the cluster is determined.
5. A new unvisited point is retrieved and processed, leading to the discovery of a further cluster or noise.
6. This process continues until all points are marked as visited.

Advantages

- This algorithm works well if all the clusters are dense enough, and they are well separated by low-density regions.
- Does not require a-priori specification of number of clusters.
- Able to identify noisy data i.e. *the outliers*, while clustering.
- DBSCAN algorithm is able to find arbitrarily size and arbitrarily shaped clusters.

Disadvantages

- DBSCAN algorithm fails in case of varying density clusters.

OPTICS (Ordering Points To Identify the Clustering Structure)

OPTICS (Ordering Points To Identify the Clustering Structure) is a density-based clustering algorithm, similar to DBSCAN (Density-Based Spatial Clustering of Applications with Noise), but it can extract clusters of varying densities and shapes. It is useful for identifying clusters of different densities in large, high-dimensional datasets.

The main idea behind OPTICS is to extract the clustering structure of a dataset by identifying the density-connected points. The algorithm builds a density-based representation of the data by creating an ordered list of points called the reachability plot. Each point in the list is associated with a reachability distance, which is a measure of how easy it is to reach that point from other points in the dataset. Points with similar reachability distances are likely to be in the same cluster.

The OPTICS algorithm follows these main steps:

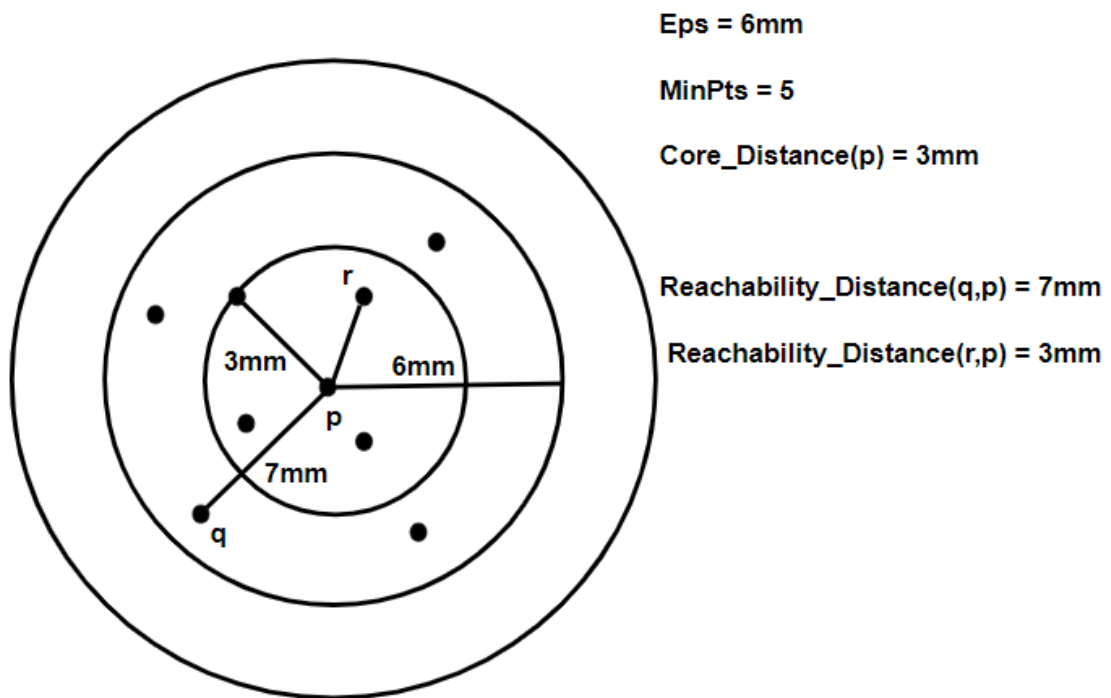
Define a density threshold parameter, ϵ , which controls the minimum density of clusters. For each point in the dataset, calculate the distance to its k -nearest neighbors. Starting with an arbitrary point, calculate the reachability distance of each point in the dataset, based on the density of its neighbors. Order the points based on their reachability distance and create the reachability plot. Extract clusters from the reachability plot by grouping points that are close to each other and have similar reachability distances. One of the main advantage of OPTICS over DBSCAN, is that it does not require to set the number of clusters in advance, instead, it extracts the clustering structure of the data and produces the reachability plot. This allows the user to have more flexibility in selecting the number of clusters, by cutting the reachability plot at a certain point.

Also, unlike other density-based clustering algorithms like DBSCAN, It can handle clusters of different densities and shapes and can identify hierarchical structure.

OPTICS is implemented in Python using the `sklearn.cluster.OPTICS` class in the scikit-learn library. It takes several parameters including the minimum density threshold (ϵ), the number of nearest neighbors to consider (`min_samples`), and a reachability distance cutoff (ξ).

They are:-

1. **Core Distance:** It is the minimum value of radius required to classify a given point as a core point. If the given point is not a Core point, then it's Core Distance is undefined.
2. **Reachability Distance:** It is defined with respect to another data point q (Let). The Reachability distance between a point p and q is the maximum of the Core Distance of p and the Euclidean Distance (or some other distance metric) between p and q . Note that The Reachability Distance is not defined if q is not a Core point.



This clustering technique is different from other clustering techniques in the sense that this technique does not explicitly segment the data into clusters. Instead, it produces a visualization of Reachability distances and uses this visualization to cluster the data.

Advantages of OPTICS:

- Can identify clusters of varying density.
- Does not require the number of clusters to be specified in advance.
- More robust to noise compared to DBSCAN.

Disadvantages of OPTICS:

- Computationally intensive for large datasets.
- Requires careful selection of parameters (ϵ and MinPts).

DENCLUE - Using Density Functions

DENSity-based **CLU**stEring by Hinneburg & Keim (KDD'98)

Major Features

- It has got a solid mathematical foundation.
- It is definitely good for data sets with large amounts of noise.
- It allows a compact mathematical description of arbitrarily shaped clusters in high-dimensional data sets.
- It is significantly faster than the existing algorithm (faster than DBSCAN by a factor of up to 45).
- But it needs a large number of parameters.

DENCLUE

It uses grid cells but only keeps information about grid cells that do actually contain data points and manages these cells in a tree-based access structure.

Influence function: This describes the impact of a data point within its neighborhood.

The Overall density of the data space can be calculated as the sum of the influence function of all data points.

The Clusters can be determined mathematically by identifying density attractors.

The Density attractors are local maxima of the overall density function.

Advantages:

- Identifies clusters of arbitrary shapes.
- Effectively handles noise.

Disadvantages:

- Sensitive to the choice of parameters.
- Computationally intensive for large datasets.
- Liable to local maxima issues.
- Primarily suited for continuous data.

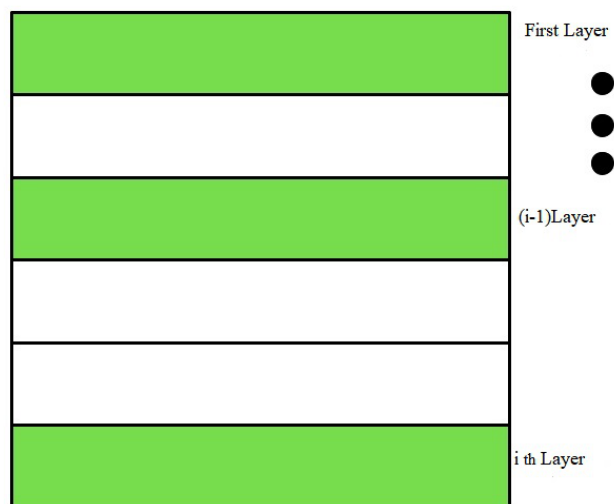
In Grid-Based Methods, the space of instance is divided into a grid structure. Clustering techniques are then applied using the Cells of the grid, instead of individual data points, as the base units. The biggest advantage of this method is to improve the processing time.

Statistical Information Grid(STING):

A STING is a grid-based clustering technique. It uses a multidimensional grid data structure that quantifies space into a finite number of cells. Instead of focusing on data points, it focuses on the value space surrounding the data points.

In STING, the spatial area is divided into rectangular cells and several levels of cells at different resolution levels. High-level cells are divided into several low-level cells.

In STING Statistical Information about attributes in each cell, such as mean, maximum, and minimum values, are precomputed and stored as statistical parameters. These statistical parameters are useful for query processing and other data analysis tasks.



The statistical parameter of higher-level cells can easily be computed from the parameters of the lower-level cells.

How STING Work:

Step 1: Determine a layer, to begin with.

Step 2: For each cell of this layer, it calculates the confidence interval or estimated range of probability that this is cell is relevant to the query.

Step 3: From the interval calculate above, it labels the cell as relevant or not relevant.

Step 4: If this layer is the bottom layer, go to point 6, otherwise, go to point 5.

Step 5: It goes down the hierarchy structure by one level. Go to point 2 for those cells that form the relevant cell of the high-level layer.

Step 6: If the specification of the query is met, go to point 8, otherwise go to point 7.

Step 7: Retrieve those data that fall into the relevant cells and do further processing. Return the result that meets the requirement of the query. Go to point 9.

Step 8: Find the regions of relevant cells. Return those regions that meet the requirement of the query. Go to point 9.

Step 9: Stop or terminate.

Advantages:

- Grid-based computing is query-independent because the statistics stored in each cell represent a summary of the data in the grid cells and are query-independent.
- The grid structure facilitates parallel processing and incremental updates.

Disadvantage:

- The main disadvantage of Sting (Statistics Grid). As we know, all cluster boundaries are either horizontal or vertical, so no diagonal boundaries are detected.

CLIQUE Algorithm:

CLIQUE Algorithm uses density and grid-based technique i.e subspace clustering algorithm and finds out the cluster by taking density threshold and a number of grids as input parameters. It is specially designed to handle datasets with a large number of dimensions. CLIQUE Algorithm is very scalable with respect to the value of the records, and a number of dimensions in the dataset because it is grid-based and uses the Apriori Property effectively. APRIORI APPROACH ?.

Apriori Approach Stated that If an X dimensional unit is dense then all its projections in X-1 dimensional space are also dense.

This means that dense regions in a given subspace must produce dense regions when projected to a low-dimensional subspace. CLIQUE restricts its search for high-dimensional dense cells to the intersection of dense cells in the subspace because CLIQUE uses apriori properties.

Working of CLIQUE Algorithm:

The CLIQUE algorithm first divides the data space into grids. It is done by dividing each dimension into equal intervals called units. After that, it identifies dense units. A unit is dense if the data points in this are exceeding the threshold value.

Once the algorithm finds dense cells along one dimension, the algorithm tries to find dense cells along two dimensions, and it works until all dense cells along the entire dimension are found.

After finding all dense cells in all dimensions, the algorithm proceeds to find the largest set ("cluster") of connected dense cells. Finally, the CLIQUE algorithm generates a minimal description of the cluster. Clusters are then generated from all dense subspaces using the apriori approach.

Advantage:

- CLIQUE is a subspace clustering algorithm that outperforms K-means, DBSCAN, and Farthest First in both execution time and accuracy.
- CLIQUE can find clusters of any shape and is able to find any number of clusters in any number of dimensions, where the number is not predetermined by a parameter.
- One of the simplest methods, and interpretability of results.

Disadvantage:

- The main disadvantage of CLIQUE Algorithm is that if the size of the cell is unsuitable for a set of very high values, then too much of the estimation will take place and the correct cluster will be unable to find.

EVALUATION OF CLUSTERS

Three important factors by which clustering can be evaluated are

(a) Clustering tendency (b) Number of clusters, k (c) Clustering quality

Clustering tendency

Before evaluating the clustering performance, making sure that data set we are working has clustering tendency and does not contain uniformly distributed points is very important. If the data does not contain clustering tendency, then clusters identified by any state of the art clustering algorithms may be irrelevant. Non-uniform distribution of points in data set becomes important in clustering.

To solve this, Hopkins test, a statistical test for spatial randomness of a variable, can be used to measure the probability of data points generated by uniform data distribution.

Plot for data from Uniform distribution

Null Hypothesis (H_0) : Data points are generated by uniform distribution (implying no meaningful clusters)

Alternate Hypothesis (H_a): Data points are generated by random data points (presence of clusters)

If $H > 0.5$, null hypothesis can be rejected and it is very much likely that data contains clusters. If H is more close to 0, then data set doesn't have clustering tendency.

Number of Optimal Clusters, k

Some of the clustering algorithms like K-means, require number of clusters, k , as clustering parameter. Getting the optimal number of clusters is very significant in the analysis. If k is too high, each point will broadly start representing a cluster and if k is too low, then data points are incorrectly clustered. Finding the optimal number of clusters leads to granularity in clustering.

There is no definitive answer for finding right number of cluster as it depends upon (a) Distribution shape (b) scale in the data set (c) clustering resolution required by user. Although finding number of clusters is a very subjective problem. There are two major approaches to find optimal number of clusters:

(1) Domain knowledge

(2) Data driven approach

Domain knowledge Domain knowledge might give some prior knowledge on finding number of clusters. For example, in case of clustering iris data set, if we have the prior knowledge of species (setosa, virginica, versicolor) , then $k = 3$. Domain knowledge driven k value gives more relevant insights.

Data driven approach If the domain knowledge is not available, mathematical methods help in finding out right number of clusters.

Empirical Method:-

A simple empirical method of finding number of clusters is Square root of $N/2$ where N is total number of data points, so that each cluster contains square root of $2 * N$

Elbow method:-

Within-cluster variance is a measure of compactness of the cluster. Lower the value of within cluster variance, higher the compactness of cluster formed.

Sum of within-cluster variance, W , is calculated for clustering analyses done with different values of k . W is a cumulative measure how good the points are clustered in the analysis. Plotting the k values and their corresponding sum of within-cluster variance helps in finding the number of clusters.

Plot of Sum of within cluster distance vs Number of clusters in Elbow method

Plot shows that number of optimal clusters = 4.

Initially, Error measure (within-cluster variance) decreases with increase in cluster number. After a particular point, $k=4$, Error measure starts flattening. Cluster number corresponding to that particular point, $k=4$, should be considered as optimal number of clusters.

Statistical approach:-

Gap statistic is a powerful statistical method to find the optimal number of clusters, k .

Similar to Elbow method, sum of within-cluster (intra-cluster) variance is calculated for different values of k .

Then Random data points from reference null distribution are generated and Sum of within-cluster variance is calculated for the clustering done for different values of k .

In Simpler words, Sum-of-within-Cluster variance of original data set for different values of k to Sum-of-within-cluster variance of reference data set (null reference data set of uniform distribution) of corresponding values of k is compared to find the ideal k value where deviation or Gap between two is highest. As Gap statistic quantifies this deviation, More the Gap statistic means more the deviation.

Gap statistic Value-Cluster number (k)

Cluster number with maximum Gap statistic value corresponds to optimal number of cluster.

Clustering quality

Once clustering is done, how well the clustering has performed can be quantified by a number of metrics. Ideal clustering is characterised by minimal intra cluster distance and maximal inter cluster distance.

There are majorly two types of measures to assess the clustering performance.

(i) Extrinsic Measures which require ground truth labels. Examples are Adjusted Rand index, Fowlkes-Mallows scores, Mutual information based scores, Homogeneity, Completeness and V-measure.

(ii) Intrinsic Measures that does not require ground truth labels. Some of the clustering performance measures are Silhouette Coefficient, Calinski-Harabasz Index, Davies-Bouldin Index etc.